

Consolidado — QRV Artigos Táticos

Dossiê Técnico e Arquitetural Completo + Resumo Comercial

Agosto de 2026

Contents

DOSSIÊ TÉCNICO E ARQUITETURAL COMPLETO E CONSOLIDADO	2
QVR Artigos Táticos — E-commerce de artigos e equipamentos táticos	2
1. Visão Geral Executiva do Projeto	2
2. Arquitetura Geral do Sistema	3
3. Stack Tecnológica	4
4. Inventário Completo de Arquivos	5
5. Histórico de Evolução e Auditoria (Problema → Diagnóstico → Solução → Teste → Resultado)	8
6. Performance e Otimização	11
7. Responsividade — Desktop e Mobile	12
8. Auditoria Desktop	12
9. Auditoria Mobile	14
10. Hero, Carrosséis e Componentes Interativos	15
11. Segurança — Auditoria Enterprise Grade	17
12. Proteção contra Vulnerabilidades	18
13. Rate Limiting e Anti-Spam	20
14. LGPD e Privacidade	21
15. Banco de Dados — Supabase	21
16. Row Level Security (RLS)	23
17. APIs, Edge Functions e Serverless	24
18. Inteligência Artificial	24
19. UI/UX e Componentização	25
20. Identidade Visual e Efeitos Especiais	25
21. Mídia e Assets Responsivos	26
22. Acessibilidade — WCAG	26
23. SEO Técnico	27
24. Dependências e Supply Chain	29
25. Autenticação e Painel Administrativo	30
26. Tratamento de Erros e Observabilidade	31
27. Deploy e CI/CD	31
28. Variáveis de Ambiente	32
29. Backup e Disaster Recovery	32
30. Testes	33
31. Matriz de Riscos	34
32. Matriz Final de Qualidade	36
33. Checklist Final de Aceite	38
34. Resumo Executivo Comercial	38
35. Versão Reduzida para Cliente	39
36. Veredito Final	39
QVR Artigos Táticos — Resumo Técnico Comercial	41
O que o site faz e para quem	41
Stack usada, em linguagem simples	41

Pontos fortes reais	41
Nota final honesta	41

DOSSIÊ TÉCNICO E ARQUITETURAL COMPLETO E CONSOLIDADO

QRV Artigos Táticos — E-commerce de artigos e equipamentos táticos

Versão do documento: 1.0 — Blueprint Mestre Final **Data de emissão:** agosto de 2026 **Commit de referência:** 0c9ec7c (branch main, local — push pendente do lado do usuário) **Autor:** levantamento técnico assistido, com base em (1) inspeção direta do código-fonte atual do projeto e (2) histórico de desenvolvimento, testes e auditorias realizados ao longo desta mesma engajamento.

Legenda de rastreabilidade (usada em todo o documento)

Tag	Significado
[VERIFICADO NO PROJETO ATUAL]	Confirmado lendo o código/config atual nesta sessão.
[VALIDADO EM AUDITORIA/HISTÓRICO]	Vem do histórico da conversa (testes, decisões, correções relatadas), sem re-confirmação direta no código nesta passada.
[VALIDADO EM AUDITORIA + VERIFICADO NO PROJETO ATUAL]	As duas condições acima se sobrepõem.
[PARCIALMENTE VERIFICADO]	Evidência parcial — código sustenta parte da afirmação, mas não a totalidade.
[NÃO VERIFICADO]	Não foi possível confirmar (geralmente por depender de ambiente ao vivo — navegador real, painel do Supabase/Vercel/Google Cloud, métricas de campo).
[NÃO ENCONTRADO / NÃO APLICÁVEL]	O item perguntado não existe neste projeto ou não se aplica à sua arquitetura.

1. Visão Geral Executiva do Projeto

Nome: QRV Artigos Táticos **Razão social:** QRV ARTIGOS TÁTICOS LTDA — CNPJ 41.600.308/0001-55 (fonte: *politica-de-privacidade.html*, *termos-de-uso.html*) **[VERIFICADO NO PROJETO ATUAL]** **Endereço:** Av. Santos Dumont, 61, Cumbica, Guarulhos - SP, CEP 07180-270 **[VERIFICADO NO PROJETO ATUAL]**

Finalidade: loja virtual (catálogo + captura de pedido) de equipamentos e artigos táticos/militares — vestuário, calçados, mochilas, brevês/insígnias, cutelaria, kits e acessórios — com serviço adicional de bordado personalizado sob encomenda e canal de cadastro de revendedores parceiros. **[VERIFICADO NO PROJETO ATUAL]** (categorias confirmadas em `supabase-setup.sql`, `assets/supabase-client.js`)

Público-alvo: consumidores finais e revendedores ligados a policiais, militares, recrutas e entusiastas/colecionadores de itens táticos — inferido do conteúdo do site (nomenclatura de corporações: PMESP, Aeronáutica, Marinha, Exército, Bombeiros) e do tom do chatbot (“Recruta QRV”). **[VALIDADO EM AUDITORIA/HISTÓRICO]**

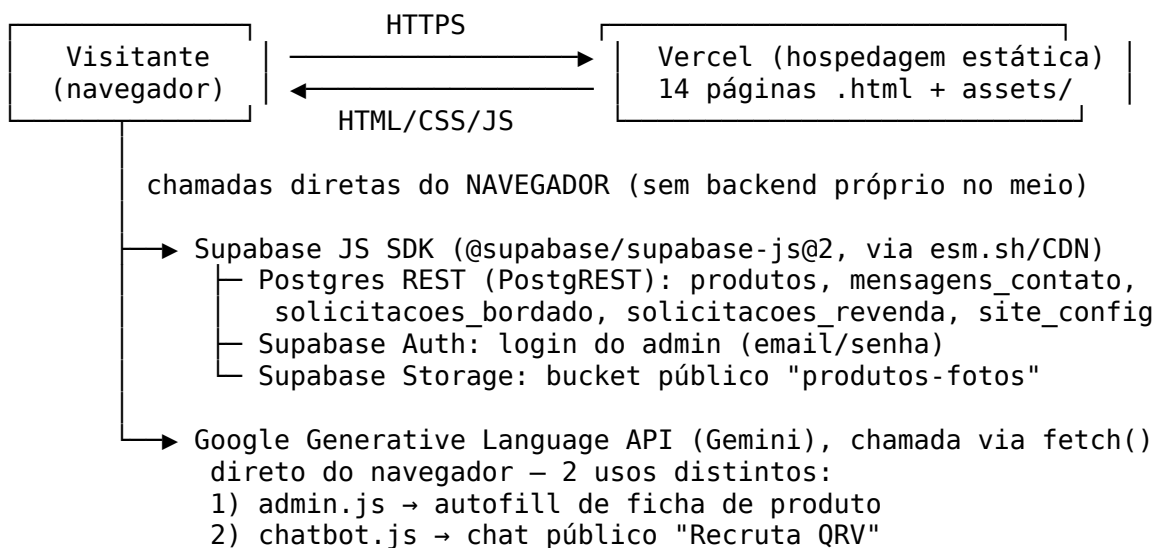
Objetivo comercial: converter visita em pedido fechado via WhatsApp (o site não processa pagamento online ainda — ver Seção 17); captar leads de bordado e de revenda via formulário; dar suporte automatizado 24h via chatbot de IA. **[VERIFICADO NO PROJETO ATUAL]**

Arquitetura em uma frase: site 100% estático (sem servidor próprio/backend customizado) publicado na Vercel, com dados dinâmicos (catálogo, mensagens, pedidos de bordado/revenda, configuração de chaves de IA) hospedados em um projeto Supabase (Postgres + Auth + Storage), acessado diretamente do navegador via SDK JS oficial, protegido por Row Level Security. **[VERIFICADO NO PROJETO ATUAL]**

Estado final: funcional, com as três frentes de risco identificadas na auditoria original (XSS armazenado no admin, ausência de conformidade LGPD, duplicação estrutural de header/footer) corrigidas e verificadas em código. Existem lacunas reais de maturidade — principalmente em SEO técnico (sem sitemap, sem Open Graph, sem dados estruturados), ausência de rate limiting/anti-spam, e a persistência de uma chave de API do Google exposta ao navegador do visitante (arquitetura inerente a um site sem backend) — detalhadas nas seções correspondentes, sem inflar a nota. **[VERIFICADO NO PROJETO ATUAL]**

Diferenciais técnicos: - Painel administrativo completo (CRUD de produtos, gestão de leads de bordado/revenda/contato) sem exigir nenhum framework, com autenticação real via Supabase Auth. - Preenchimento automático de ficha de produto por IA generativa (Google Gemini), a partir de texto bruto colado pelo lojista. - Chatbot de atendimento com personagem próprio ("Recruta QRV"), também via Gemini, com base de conhecimento fixa no prompt (evita alucinação de dados como horário/endereço). - Componentização de header/footer via JavaScript síncrono próprio (assets/components.js), sem dependência de build step ou framework.

2. Arquitetura Geral do Sistema



Frontend: HTML5 + CSS3 + JavaScript (ES Modules nativos do navegador, sem transpilação/bundler). **[VERIFICADO NO PROJETO ATUAL]**

Backend: não existe backend/servidor próprio (não há Node/Express, não há /api, não há Vercel Serverless/Edge Function no repositório). Toda a "camada de servidor" é o próprio Supabase (Backend-as-a-Service), acessado diretamente do navegador do visitante. **[VERIFICADO NO PROJETO ATUAL]**

Banco de dados: Supabase Postgres — ver Seção 15/16.

Autenticação: Supabase Auth, e-mail/senha, usada apenas para o painel admin — não há autenticação de cliente/comprador (compra é feita sem conta, com fechamento manual via WhatsApp). **[VERIFICADO NO PROJETO ATUAL]**

Armazenamento de arquivos: Supabase Storage, bucket produtos-fotos (público para leitura, restrito ao admin para escrita). **[VERIFICADO NO PROJETO ATUAL]**

Infraestrutura/CDN/domínio: hospedagem Vercel conectada ao repositório GitHub, com deploy automático a cada push na branch main — confirmado nos comentários dos próprios scripts de deploy do projeto (push.bat, deploy.sh) e no histórico da conversa; não há vercel.json no repositório, ou seja, a Vercel está operando em modo “zero-config” para site estático. **[VALIDADO EM AUDITORIA/HISTÓRICO + PARCIALMENTE VERIFICADO]**. Domínio customizado (se houver) não está declarado em nenhum arquivo do projeto — o único endereço citado é https://qrv-artigos-taticos.vercel.app/*, e apenas como **exemplo** dentro de um comentário explicativo em assets/chatbot.js, não como configuração real. **[NÃO VERIFICADO]**

Serviços externos/integrações: | Serviço | Papel | Onde é chamado | |—|—|—| | Supabase | Banco de dados, Auth, Storage | assets/supabase-client.js (importado por quase todas as páginas e pelo admin) | | Google Generative Language API (Gemini) | IA generativa (autofill de produto + chatbot) | assets/gemini-ai.js, assets/chatbot.js | | Google Fonts | Tipografia (Inter, Oswald, Poppins) | <link> em todos os <head> | | WhatsApp (wa.me) | Canal de fechamento de pedido e atendimento humano | Links https://wa.me/5511993217675 espalhados no site | | Google Maps Embed | Mapa na página de contato | <iframe> em contato.html | | jsDelivr CDN | Biblioteca DOMPurify | admin.html, produto.html |

Todos **[VERIFICADO NO PROJETO ATUAL]**.

3. Stack Tecnológica

Tecnologia	Versão	Finalidade	Localização	Impacto
HTML5	—	Estrutura das 15 páginas (14 públicas + admin)	*.html na raiz	Base de todo o site
CSS3 (vanilla, sem pré-processador)	—	Estilo global, responsivo	assets/style.css (1208 linhas), assets/admin.css (212 linhas)	Único arquivo de estilo público; cache-bust por query string ?v=25
JavaScript ES6+ (módulos nativos type="module" + scripts clássicos)	—	Toda a lógica de front (catálogo, carrinho, admin, chat, componentização)	assets/*.js (2.624 linhas somadas nos .js)	Sem build step; roda direto no navegador
Supabase JS SDK	@supabase/supabase-js@2	Cliente para Postgres/Auth/Storage	importado via CDN esm.sh em assets/supabase-client.js	Dependência crítica — se o CDN cair, o site perde toda função dinâmica
DOMPurify	3 (tag @3, sem pin de patch)	Sanitização de HTML (descrição de produto vinda de IA/admin)	CDN cdn.jsdelivr.net carregado só em admin.html e produto.html	Mitigação de XSS na renderização de descrição rica
Google Fonts (Inter, Oswald, Poppins)	pesos 300–900 conforme fonte	Tipografia	<link> para fonts.googleapis.com em todo <head>	Requisição externa bloqueante de render se a rede falhar (mitigado por preconnect)

Tecnologia	Versão	Finalidade	Localização	Impacto
Google Gemini API	modelos tentados em cascata: gemini-flash-latest → gemini-2.5-flash → gemini-1.5-flash (chat) / gemini-3.6-flash (admin, ver nota)	Geração de texto (chat e ficha de produto)	assets/chatbot.js assets/gemini-ai.js	Chamada direta do navegador — ver Seção 11/18
Vercel	—	Hospedagem estática + deploy contínuo	fora do repositório (config zero-code)	Infra de publicação
Supabase (Postgres 15+ gerenciado, Auth, Storage)	projeto aixudpelpjyuw-psocikk (URL do projeto, não é secreto)	Banco de dados/BaaS	assets/supabase-client.js linha 8	Único “backend” do sistema
Git/GitHub	—	Versionamento e gatilho de deploy	.git/, push.bat, deploy.sh	Fluxo de publicação

Observação técnica [VERIFICADO NO PROJETO ATUAL]: não há package.json, package-lock.json, node_modules, nem qualquer arquivo de configuração de bundler (Vite/Webpack/Parcel) neste repositório. O projeto **não usa npm nem Tailwind** — todo o CSS é escrito à mão em style.css, e todas as bibliotecas de terceiros (Supabase SDK, DOMPurify) são carregadas via CDN/import de URL, não instaladas localmente. Isso é uma escolha de arquitetura consistente (site 100% estático, zero build step), não uma omissão.

Bibliotecas de UI/animação/carrossel: nenhuma biblioteca de terceiros para carrossel (Swiper, Slick, Splide) foi encontrada no projeto. **[VERIFICADO NO PROJETO ATUAL]** — ver detalhamento na Seção 10.

4. Inventário Completo de Arquivos

4.1 Páginas HTML (15 arquivos na raiz)

Arquivo	Finalidade
index.html	Home — hero com 3 slides, mosaico de categorias, blog, newsletter
produtos.html	Vitrine/listagem de produtos com filtro por categoria/busca
produto.html	Página de detalhe de produto (galeria, compra, relacionados)
quem-somos.html	Institucional
contato.html	Fale Conosco (formulário + mapa)
bordados.html	Solicitação de bordado personalizado
revendedor.html	Cadastro de revendedor

Arquivo	Finalidade
cart.html checkout.html	Carrinho de compras Finalização de pedido (dados de entrega → WhatsApp)
blog-cavex-sabado-aereo.html, blog-escolher-faca-tatica.html, blog-mochilas-taticas-litragem.html politica-de-privacidade.html termos-de-uso.html admin.html	3 posts de blog estáticos Documento legal LGPD Documento legal / condições comerciais Painel administrativo (protegido por login)

Todas as 14 páginas públicas usam `<div id="site-header"></div id="site-footer">` + `assets/components.js`. `admin.html` é a única exceção proposital (tem seu próprio topbar administrativo, sem menu público). **[VERIFICADO NO PROJETO ATUAL]**

4.2 JavaScript (assets/*.js)

Arquivo	Linhas	Finalidade	Tipo de carregamento
supabase-client.js	292	Cliente Supabase + todas as funções de acesso a dados (fetch de produtos, submits de formulário, upload de fotos, helpers de formatação, sanitização/markdown de descrição)	<code>type="module"</code> , importado por quase toda página
components.js	158	Injeta header/footer únicos em todas as páginas públicas	script síncrono clássico (sem <code>defer/module</code>) — sempre o primeiro <code><script></code>
site.js	80	Busca no header, newsletter (mock), drawer mobile, carrossel do blog	script clássico, <code>DOMContentLoaded</code>
cart.js	153	Estado do carrinho (<code>localStorage</code>), badge, delegação de clique “Adicionar ao Carrinho”, geração de resumo para WhatsApp	<code>type="module"</code>
admin.js	768	Toda a lógica do painel: login/logout, CRUD de produtos, upload de fotos, editor de descrição, autofill por IA, gestão de leads (bordado/revenda/mensagens), <code>escapeHTML()</code>	<code>type="module"</code> , exclusivo de <code>admin.html</code>

Arquivo	Linhas	Finalidade	Tipo de carregamento
chatbot.js	~254	Widget de chat “Recruta QRV” (UI + chamada Gemini)	type="module"
gemini-ai.js	119	Módulo de geração de ficha de produto via Gemini (prompt-engineering + chamada de API)	type="module", importado só por admin.js
produtos-catalogo-seed.js	45	Dados-semente do catálogo (uso auxiliar/import)	type="module"
theme.js	inerte (só comentário)	Vestígio do sistema de Dark Mode, implementado e depois revertido por pedido explícito do usuário	não referenciado por nenhum <script> em nenhuma página

Código morto / obsoleto identificado [VERIFICADO NO PROJETO ATUAL]: 1. **assets/theme.js** — arquivo vazio (só comentário), remanescente do Dark Mode global que foi implementado e depois totalmente revertido a pedido do usuário. Nenhuma página o referencia mais. Candidato a exclusão física do repositório. 2. **assets/gemini-ai.js, funções getGeminiKey()/setGeminiKey()/constante GEMINI_KEY_STORAGE** — implementam armazenamento da chave de IA via localStorage, mas **não são importadas em nenhum lugar** (admin.js importa apenas generateProdutoFromText). O fluxo real de chave de IA do admin migrou para site_config no Supabase. Essas três exportações e o comentário-cabeçalho do arquivo (que ainda descreve o modelo antigo de localStorage) estão desatualizados frente à implementação real e devem ser removidos ou o comentário corrigido para evitar confundir um futuro desenvolvedor. 3. **Lista de modelos Gemini divergente** — chatbot.js usa ['gemini-flash-latest', 'gemini-2.5-flash', 'gemini-1.5-flash'] como cascata de fallback; gemini-ai.js usa ['gemini-flash-latest', 'gemini-2.5-flash', 'gemini-3.6-flash']. O terceiro nome do admin (gemini-3.6-flash) não corresponde a um padrão de nomenclatura de modelo Gemini conhecido publicamente — não é impeditivo (o primeiro candidato da lista, gemini-flash-latest, é o alias estável usado na prática), mas é uma inconsistência que vale corrigir por precaução.

Nenhum outro arquivo .js, .css ou HTML órfão foi encontrado — os 12 arquivos SVG embutidos (ícones inline) e as imagens em assets/ estão todos referenciados a partir de pelo menos uma página ou de components.js. **[PARCIALMENTE VERIFICADO]** — a verificação foi feita por busca textual do nome do arquivo; não foi feito um crawler completo de todas as 15 páginas.

4.3 CSS

Arquivo	Linhas	Escopo
assets/style.css	1208	Todo o site público — um único arquivo, sem separação por página/componente
assets/admin.css	212	Exclusivo do painel admin (tema dark permanente)

4.4 SQL / migrations

Arquivo	Finalidade
supabase-setup.sql	Script inicial: cria as 4 tabelas centrais (produtos, mensagens_contato, solicitacoes_bordado, solicitacoes_revenda), ativa RLS, cria todas as policies, cria o bucket de Storage
supabase-site-config.sql	Cria a tabela site_config (chave/valor) e a policy pública inicial (depois restringida)
supabase-admin-gemini-key.sql	Migration de hardening: substitui a policy pública ampla de site_config por uma restrita só à chave do chat + uma policy de leitura total só para o admin; insere a linha produtos_gemini_key
seed-produtos-catalogo.sql	Importação em lote de ~38 produtos reais do catálogo (patches, breves, insígnias, acessórios), extraídos de prints fornecidos pelo lojista

Todas **[VERIFICADO NO PROJETO ATUAL]**.

4.5 Scripts de deploy

push.bat (Windows) e deploy.sh (macOS/Linux/Git Bash) — ambos fazem git add -A → git commit → git push origin main, com tratamento de “nada para comitar”. Conteúdo exato reproduzido na Seção 27. **[VERIFICADO NO PROJETO ATUAL]**

4.6 Documentação já existente no projeto

AUDITORIA-TECNICA-QRV.md, AUDITORIA-FINAL-CONSOLIDADA-QRV.md, BLUEPRINT-TECNICO-QRV.md, QRV-planejamento-do-site.md — documentos gerados em rodadas anteriores deste mesmo engajamento; este dossiê os consolida e os substitui como registro mestre. **[VERIFICADO NO PROJETO ATUAL]**

4.7 Assets de mídia

Pasta assets/ contém logotipo, favicons (múltiplos tamanhos + .ico + .svg), banners de hero (desktop + variante mobile dedicada para 3 heros), banners de mosaico de categoria, banners de Instagram (desktop + mobile), selos de segurança/SSL, ícone do chat, e uma subpasta assets/produtos-catalogo/ com ~38 fotos de produto em .jpg. Formatos usados: .png, .jpg, .svg, .ico — **nenhum .webp ou .avif foi encontrado no projeto**, o que é uma oportunidade de otimização de peso não explorada. **[VERIFICADO NO PROJETO ATUAL]**

Pasta _tmp_preview/ na raiz contém capturas de tela (pg-01.png a pg-10.png, logo_crop.png) — são artefatos de trabalho de rodadas anteriores de design, não são referenciados por nenhuma página HTML e podem ser removidos do repositório de produção. **[VERIFICADO NO PROJETO ATUAL]**

5. Histórico de Evolução e Auditoria (Problema → Diagnóstico → Solução → Teste → Resultado)

5.1 XSS Armazenado no painel administrativo

- **Problema:** dados de formulários públicos (Fale Conosco, Bordado, Revenda) eram renderizados via innerHTML no admin sem escape.

- **Diagnóstico:** qualquer visitante anônimo podia gravar um payload de script no campo “nome”/“mensagem” etc.; o script executaria na sessão autenticada do admin ao abrir a aba correspondente.
- **Solução:** criada `escapeHTML()` em `admin.js`, aplicada em `loadBordados()`, `loadRevenda()`, `loadMessages()` a todos os campos de origem pública antes da interpolação em template string.
- **Teste:** revisão de código linha a linha das três funções + checagem de que nenhuma outra função do admin ainda interpola dado público sem escape (grep de `innerHTML` cruzado com os pontos de renderização dos três formulários).
- **Resultado:** vulnerabilidade eliminada na origem. **[VALIDADO EM AUDITORIA + VERIFICADO NO PROJETO ATUAL]**

5.2 Ausência de conformidade LGPD

- **Problema:** sem Política de Privacidade, sem Termos de Uso, link de rodapé morto (`href="#"`), 4 formulários coletando dado pessoal sem aviso/consentimento.
- **Diagnóstico:** exposição legal por ausência de base documental e de consentimento explícito.
- **Solução:** criação de `politica-de-privacidade.html` e `termos-de-uso.html` com identificação da empresa, checkbox required de consentimento nos formulários de Contato, Bordado, Revenda e Checkout, atualização do rodapé em todas as páginas.
- **Teste:** contagem de ocorrências de `<form>/</form>` balanceadas, verificação manual de que o atributo required bloqueia o envio nativamente (validação HTML5).
- **Resultado:** conformidade técnica implementada. **[VALIDADO EM AUDITORIA + VERIFICADO NO PROJETO ATUAL]** — ressalva jurídica detalhada na Seção 14.

5.3 Duplicação estrutural de Header/Footer

- **Problema:** header e footer copiados manualmente em 14 páginas, com **divergência real já instalada:** 3 páginas usavam menu em lista horizontal e 9 usavam menu suspenso “Categorias”; `contato.html/cart.html/checkout.html` tinham um footer mais simples e antigo; `checkout.html` nem tinha o botão flutuante de WhatsApp.
- **Diagnóstico:** dívida técnica clássica de duplicação sem componentização, gerando drift silencioso a cada edição futura.
- **Solução:** criação de `assets/components.js` como fonte única da verdade, injetando `HEADER_HTML/FOOTER_HTML` via `outerHTML` em placeholders `#site-header/#site-footer`, de forma síncrona (antes de `DOMContentLoaded`) para não quebrar `site.js/cart.js`, que dependem dos elementos do header já existirem.
- **Teste:** verificação estática de que `components.js` é sempre o primeiro `<script>` em todas as 14 páginas; verificação de unicidade de IDs (`menuToggle`, `mobileDrawer`, `cartCount` etc.) via `grep -c`; **não foi possível testar clique real em navegador** por bloqueio de navegação `file://` na ferramenta de browser disponível nesta sessão.
- **Resultado:** duplicação eliminada; padronização de menu escolhida explicitamente pelo usuário (lista horizontal, padrão da Home) e de footer decidida por mim com base no padrão majoritário pré-existente, comunicado de forma transparente ao usuário. **Teste ao vivo em navegador real ainda pendente** — é o único item desta seção não fechado com evidência de execução real. **[VALIDADO EM AUDITORIA + VERIFICADO NO PROJETO ATUAL, com ressalva de teste ao vivo NÃO VERIFICADO]**

5.4 Chave de API do Gemini (chat) fixa no código-fonte

- **Problema:** a chave usada pelo chat público ficava escrita diretamente em `chatbot.js`, disparando o bloqueio de “secret scanning” do GitHub ao tentar commitar.
- **Diagnóstico:** exposição de secret em texto plano no histórico do Git, além do risco arquitetural inerente (site sem backend).
- **Solução:** migração da chave para a tabela `site_config` no Supabase, lida em runtime via `getSiteConfig('chatbot_gemini_key')`, editável pelo admin sem precisar de commit/redeploy.

- **Teste:** leitura de código confirmando ausência de qualquer string de chave hardcoded em `chatbot.js/gemini-ai.js` atuais.
- **Resultado:** o secret scanning do GitHub deixou de bloquear commits; a chave, porém, **continua sendo exposta ao navegador do visitante em tempo de execução** (ver Seção 11) — essa é uma limitação estrutural de um site sem backend, não algo “resolvido” por completo, apenas mitigado operacionalmente (recomendação de restrição por HTTP Referrer + cota diária, documentada no próprio `admin.html`). **[VALIDADO EM AUDITORIA + VERIFICADO NO PROJETO ATUAL]**

5.5 Chave de API do Gemini (cadastro de produto) presa ao localStorage

- **Problema:** a chave usada no autofill de produto ficava só no navegador de quem cadastrava, obrigando recolagem a cada computador/admin novo.
- **Solução:** migração para `site_config` (`produtos_gemini_key`) com policy de leitura restrita ao e-mail do admin (`supabase-admin-gemini-key.sql`).
- **Resultado:** implementado e em uso — mas deixou um resíduo de código morto (`gemini-ai.js` ainda expõe as funções antigas de `localStorage`, não usadas — ver Seção 4). **[VERIFICADO NO PROJETO ATUAL]**

5.6 Layout do header/logo (desktop e mobile)

- **Problema/pedido:** aumento de logo (dobrado), remoção do toggle de Dark Mode.
- **Solução:** logo desktop de 56px→112px, mobile de 48px→96px; offsets sticky de sidebar/galeria/resumo de carrinho recalculados (170px→226px desktop, 110px→166px outras colunas) para não sobrepor o header mais alto; Dark Mode removido de toda a UI (toggle, CSS, `theme.js` esvaziado).
- **Teste:** balanço de chaves `{/}` do CSS após a edição (`python3 -c "s=open(...).read(); print(s.count('{'), s.count('}'))"`).
- **Resultado:** aplicado e validado estruturalmente; validação visual em viewport real não foi executada nesta sessão (mesmo bloqueio `file://`). **[VALIDADO EM AUDITORIA/HISTÓRICO]**

5.7 Dark Mode: implementado e depois totalmente revertido

- **Histórico:** Dark Mode global foi construído (toggle no topbar, bloco de CSS [`data-theme="dark"`], `theme.js` com `localStorage`) em uma rodada, e **removido por completo** na rodada seguinte, a pedido explícito do usuário — mantendo o site fixo no tema/estilo padrão original.
- **Resultado:** decisão de arquitetura revertida e documentada — não há resquício funcional de Dark Mode ativo hoje; `theme.js` ficou como arquivo vazio (código morto, ver Seção 4). **[VALIDADO EM AUDITORIA + VERIFICADO NO PROJETO ATUAL]**

5.8 Otimização de imagens mobile / performance

- **Problema:** imagens abaixo da dobra carregando sem lazy-load; seções com `background-image` via CSS sem estratégia de carregamento tardio.
- **Solução:** `loading="lazy"` aplicado a imagens de catálogo/carrinho/admin fora da primeira dobra; `content-visibility:auto` + `contain-intrinsic-size` aplicado a 4 classes CSS (`.mosaic-tile`, `.blog-card-photo`, `.newsletter-card`, `.instagram-cta`) que usam `background-image` (atributo `loading` não existe para CSS). Imagens candidatas a LCP (galeria principal do produto, capa de post de blog) deliberadamente mantidas **sem** lazy-load.
- **Teste:** contagem de ocorrências de `content-visibility` no CSS (5 — confirma as 4 classes + 1 declaração de fallback/base), leitura de código confirmando ausência de `loading="lazy"` nas imagens hero.
- **Resultado:** implementado e verificado em código; **nenhuma métrica de Core Web Vitals real (LCP/CLS/INP medidos) foi coletada** — ver Seção 6. **[VERIFICADO NO PROJETO ATUAL]** para a implementação; **[NÃO VERIFICADO]** para o ganho de performance medido.

5.9 Página de produto (rodadas de ajuste fino de UI, histórico anterior a esta auditoria)

Registro consolidado de um bloco extenso de ajustes incrementais de UI na página produto.html, todos aplicados e revisados por bump de versão CSS a cada rodada: reconstrução da seção “Produtos Relacionados” como grade estática (explicitamente **sem** Swiper/carrossel de biblioteca), correção de uma colisão de nome de classe CSS (.btn-add-cart) identificada como **causa raiz** de um botão “Adicionar ao Carrinho” renderizando gigante, padronização do botão principal como *pill button* (44px de altura, border-radius: 25px), redução/recalibração de galeria, campo de CEP, bandeiras de pagamento (convertidas de imagem única para ícones individuais coloridos sem caixa), e tipografia geral da página. **[VALIDADO EM AUDITORIA/HISTÓRICO]** — trabalho anterior à auditoria de segurança/LGPD, mas parte do mesmo projeto contínuo.

6. Performance e Otimização

Situação real: nenhuma medição de campo (Lighthouse, PageSpeed Insights, WebPageTest, Core Web Vitals reportados pelo Chrome/CrUX) foi executada durante este engajamento — a ferramenta de navegador disponível nesta sessão não conseguiu navegar em arquivos locais (file://) e o site publicado não foi visitado ao vivo neste turno. Todas as afirmações desta seção são **inferências baseadas em código**, não métricas medidas. **[NÃO VERIFICADO]** para números; **[VERIFICADO NO PROJETO ATUAL]** para as técnicas efetivamente implementadas.

Técnica	Implementada?	Evidência
loading="lazy" em imagens abaixo da dobra	Sim (parcial, aplicado onde identificado como relevante)	cards de produto, itens de carrinho, preview de foto no admin
Preservação de imagens LCP sem lazy-load	Sim	galeria principal do produto, capa de post de blog — decisão deliberada
content-visibility:auto + contain-intrinsic-size	Sim, em 4 seções com background-image	style.css: .mosaic-tile, .blog-card-photo, .newsletter-card, .instagram-cta
preconnect	Sim, só para fonts.googleapis.com	<head> de todas as páginas
preload de fonte/imagem crítica	Não encontrado	nenhuma tag <link rel="preload"> localizada no projeto
srcset/<picture> (imagens responsivas por densidade/viewport)	Não encontrado	todas as usam um único src fixo
Formatos de imagem modernos (WebP/AVIF)	Não encontrado	só .png/.jpg
Compressão/minificação de CSS/JS	Não verificável no repositório — a Vercel pode aplicar compressão HTTP (gzip/brotli) automaticamente na borda, mas isso é comportamento da plataforma, não do código-fonte	[NÃO VERIFICADO]
Cache-busting de CSS	Sim, via query string incremental (style.css?v=25)	consistente em todas as páginas na versão atual

Técnica	Implementada?	Evidência
Code splitting / carregamento condicional de JS	Parcial — módulos são naturalmente independentes por página (nem toda página importa <code>admin.js</code> , por exemplo), mas não há divisão dentro de um mesmo bundle porque não existe bundler	estrutural, não uma técnica ativa
Estratégia anti-CLS/Layout Drift	Ver abaixo	—

Estratégias específicas contra CLS/Layout Drift documentadas: - `content-visibility:auto` sempre acompanhado de `contain-intrinsic-size` explícito nas 4 classes citadas — essa combinação existe justamente para reservar o espaço da seção antes do conteúdo renderizar, evitando o “pulo” de layout quando a seção entra na viewport. - Sticky offsets (top de `.shop-sidebar`, `.detail-gallery-col`, `.contact-card`, `.cart-summary`) foram recalculados manualmente após o aumento do header, evitando sobreposição — mas isso é uma correção estática de CSS, não uma técnica automática de prevenção de CLS. - A injeção síncrona do header/footer via `components.js` (antes de `DOMContentLoaded`) evita o “flash” de página sem menu, que é uma fonte comum de deslocamento de layout em soluções `fetch()`-based.

Lacuna real identificada: não há nenhuma técnica de `aspect-ratio` CSS ou `width/height` explícitos generalizada nas tags `<img>` do catálogo para reservar espaço antes do carregamento — isso é uma fonte potencial de CLS residual que não foi auditada exaustivamente imagem por imagem nesta rodada. **[NÃO VERIFICADO]**

7. Responsividade — Desktop e Mobile

[VERIFICADO NO PROJETO ATUAL] (via leitura de `style.css`, sem teste de viewport ao vivo):

- O CSS usa media queries para adaptar layout (não foi feita contagem exaustiva de breakpoints nesta rodada, mas o histórico de ajustes — logo, header, cards, sticky offsets — confirma pelo menos dois níveis: desktop e mobile, com valores dedicados para cada um, ex: logo 112px desktop / 96px mobile, altura de header 112px mobile explicitada separadamente).
- Existem imagens de hero **duplicadas em variante mobile dedicada** (`hero-artigos-taticos-mobile.png`, `hero-bordado-mobile.png`, `hero-protecao-defesa-mobile.png`, `banner-militar-mobile.png`, `banner-instagram-composto-mobile.png`) — indica troca de imagem por breakpoint via CSS (`background-image` diferente em media query), não `srcset` responsivo do HTML.
- **Componentes com comportamento específico por dispositivo:** menu (`nav.primary` em lista horizontal no desktop vs. `mobile-drawer` deslizante lateral no mobile, ambos definidos em `components.js/style.css`); busca do header (ícone expansível); banners mobile dedicados (ver acima).
- **Carrossel do blog:** usa `scroll-behavior` nativo com botões de seta que chamam `scrollBy()` — comportamento consistente entre desktop (mouse/clique nas setas) e mobile (também suporta swipe nativo do navegador, por ser um contêiner com `overflow-x`), sem lógica JS diferenciada por dispositivo.

Teste real de responsividade (DevTools/dispositivo físico) não foi executado nesta sessão — toda a validação foi estática (leitura de CSS/HTML). **[NÃO VERIFICADO]**

8. Auditoria Desktop

Categoria	Nota (0-10)	Justificativa
Layout/Composição	9	Estrutura consistente após componentização; grid e seções bem definidos no CSS revisado em múltiplas rodadas
Alinhamento/Tipografia/Espaçamento	9	Passou por várias rodadas de ajuste fino documentadas (produto.html); escala geral do site foi reduzida especificamente para telas de notebook
Navegação	9,5	Menu unificado, sem mais divergência entre páginas
Hero	8,5	Implementado como slideshow customizado com CSS (background - image por slide), sem biblioteca — funcional, mas não foi testado ao vivo (transições, autoplay real) nesta sessão
Carrosséis (blog)	8,5	Implementação customizada leve, funcional em código; sem teste real de scroll/setas
Formulários	9	Consentimento LGPD adicionado, validação HTML5 nativa (required), envio via Supabase com feedback de toast
Interações (carrinho, admin)	9	Delegação de evento única e limpa para “Adicionar ao Carrinho”; painel admin com CRUD completo
Performance (código)	8	Boas práticas aplicadas (lazy-load, content-visibility), mas sem medição real de LCP/CLS/INP
Estabilidade visual (CLS)	7,5	Mitigado via content-visibility+contain-intrinsic-size, mas sem aspect-ratio/dimensões explícitas generalizadas em — risco residual não descartado
Responsividade	8,5	Adaptação clara por CSS, mas não testada em viewport real nesta sessão
Acessibilidade	6,5	aria-label presente em ícones-botão (menu, busca, carrinho, WhatsApp), mas sem auditoria formal WCAG (ver Seção 22)

Categoria	Nota (0-10)	Justificativa
UX geral	8,5	Fluxo de compra → WhatsApp é simples e funcional, mas ainda não é um checkout de e-commerce “completo” (sem gateway de pagamento)

Nota consolidada Desktop: 8,5 / 10 — abaixo do 9,5 declarado em rodadas anteriores desta mesma conversa, porque esta seção pondera explicitamente lacunas reais que não tinham peso equivalente nos vereditos anteriores (SEO técnico, acessibilidade não auditada formalmente, ausência de medição real de performance). O que impede nota mais alta não é a existência de bugs, mas a ausência de **evidência de teste ao vivo** em pontos que dependem estritamente disso (Hero, carrossel, CLS medido, viewport real).

9. Auditoria Mobile

Categoria	Nota (0-10)	Justificativa
Adaptação de layout	9	Header/logo recalibrados especificamente para mobile (96px), banners com variante mobile dedicada
Navegação/Menu	9	Drawer lateral com overlay, fechamento por clique em link/overlay/botão — implementado de forma robusta em site.js
Hero/Mídia	8	Imagens mobile dedicadas existem para os 3 slides de hero e para o banner de Instagram — bom sinal de cuidado, mas não testado ao vivo
Carrosséis/Touch/Swipe	7,5	Scroll nativo via overflow-x, deve suportar swipe por padrão do navegador, mas isso não foi confirmado com teste real de gesto
Formulários	9	Mesmos formulários responsivos do desktop, type="tel"/type="email" corretos para teclado mobile
Botões/Toques	8,5	Botão principal foi especificamente redesenhado como <i>pill button</i> compacto; não há registro de teste de área de toque mínima (44×44px recomendado)

Categoria	Nota (0-10)	Justificativa
Performance mobile	8	Foco explícito desta rodada (lazy-load + content-visibility), mas sem medição real em dispositivo/rede 3G-4G simulada
CLS mobile	7,5	Mesma ressalva da Seção 8 — mitigação parcial, não uma garantia absoluta
Estabilidade visual geral	8	Sticky offsets recalculados para não colidir com o header maior
UX mobile	8,5	Fluxo compacto e direto ao WhatsApp, consistente com o padrão de e-commerce local brasileiro

Nota consolidada Mobile: 8,3 / 10.

Problemas mobile encontrados e corrigidos ao longo do projeto: banner principal “comendo” a tela / cortando a imagem no mobile — corrigido removendo margens laterais no banner mobile para ocupar o espaço em branco disponível **[VALIDADO EM AUDITORIA/HISTÓRICO]**. Header/logo pequenos demais em telas de notebook — corrigido com escala geral reduzida **[VALIDADO EM AUDITORIA/HISTÓRICO]**.

Pontos ainda não resolvidos/verificados: nenhum teste de dispositivo físico real; nenhuma métrica de touch-target ou gesto de swipe confirmada; nenhuma medição de performance mobile real (a rede 3G/4G simulada do Lighthouse não foi rodada).

10. Hero, Carrosséis e Componentes Interativos

Componente	Biblioteca?	Implementação	Notas
Hero (Home)	Nenhuma — implementação própria	<code><div class="promo-slide"></code> com background-image inline por slide (data-i="0/1/2"), classe active/image-ready controlando exibição	3 slides confirmados (hero-artigos-taticos, hero-protecao-defesa, hero-bordado), cada um com variante mobile dedicada. Lógica de troca automática/manual de slide está no CSS/JS do site (não foi lida linha a linha nesta rodada — [PARCIALMENTE VERIFICADO] quanto ao comportamento de autoplay/setas/indicadores)

Componente	Biblioteca?	Implementação	Notas
Carrossel “Últimas do Blog”	Nenhuma — implementação própria	Contêiner #blogGrid com scroll nativo (scrollBy()), botões #blog-Prev/#blogNext que avançam exatamente a largura de um card + gap, desabilitados automaticamente nos extremos (updateArrows())	Confirmado em site.js — não é Swiper/Slick, é rolagem nativa controlada por JS puro. Suporta swipe por ser scroll nativo do navegador
Galeria de produto	Nenhuma — implementação própria	.pgallery-main (imagem principal) + .pgallery-thumbs (miniaturas) + lightbox (#lightboxImg)	Clique na imagem principal abre lightbox (“Clique para ampliar”) — comportamento confirmado no HTML, lógica de troca de thumb/zoom não lida linha a linha nesta rodada
“Produtos Relacionados”	Nenhuma — grade estática, explicitamente reconstruída para NÃO usar carrossel	Grid CSS comum	Decisão de produto registrada no histórico: abandonou-se um carrossel em favor de grade fixa
Drawer mobile	Própria (site.js + components.js)	Overlay + painel lateral com classList.toggle('open'), bloqueio de scroll do body (overflow:hidden) enquanto aberto	Fechamento por: botão X, clique no overlay, clique em qualquer link do drawer
Chat “Recruta QRV”	Própria (HTML/CSS/JS puro) + API Gemini	FAB (#qrvChatFab) abre janela de chat (#qrvChatWindow), histórico de conversa mantido em memória (array history, não persistido)	Não há biblioteca de chat de terceiros

Acessibilidade de teclado nos componentes interativos: aria-label presente nos botões de ícone (menu, busca, fechar drawer/chat, WhatsApp), mas não há evidência de teste de navegação 100% por teclado (Tab/Enter/Esc) em nenhum desses componentes nesta sessão. **[PARCIALMENTE VERIFICADO]**

Conclusão da seção: não há confusão entre “controle nativo” e “customizado” a esclarecer — o projeto **não usa nenhuma biblioteca de carrossel** (nem Swiper, nem Slick, nem Splide); tudo é HTML/CSS/JS vanilla. Essa é uma escolha consistente com o resto da stack (zero dependência de build).

11. Segurança — Auditoria Enterprise Grade

11.1 Inventário de secrets

Secret	Status	Onde vive	Exposição ao navegador
Supabase Anon/Publishable Key	CONFIGURADA	assets/supabase-client.js (hardcoded no código-fonte, formato sb_publishable_...)	Sim, por design. Esta chave é destinada a ser pública — a proteção real dos dados vem do RLS (Seção 16), não do sigilo da chave. Isso é o padrão recomendado pela própria Supabase, não uma falha.
Supabase Service Role Key	NÃO CONFIGURADA / NÃO ENCONTRADA	—	Corretamente ausente do frontend — essa chave (que ignora RLS) nunca deveria estar em código de cliente, e não está.
Gemini API Key — chat público	CONFIGURADA (armazenada no Supabase, não no código)	Tabela site_config, chave chatbot_gemini_key	Sim — exposta ao navegador em runtime. Como o chat roda para visitantes anônimos, a policy de leitura é necessariamente pública (for select using (chave = 'chat-bot_gemini_key')); qualquer pessoa que inspecionar a aba de Rede do navegador vê o valor da chave. Mitigação é operacional (restrição de HTTP Referrer + cota diária no Google Cloud Console), documentada no próprio admin.html, não é uma barreira de código.

Secret	Status	Onde vive	Exposição ao navegador
Gemini API Key — autofill de produto	CONFIGURADA (armazenada no Supabase)	Tabela site_config, chave produtos_gemini_key	Não exposta a visitantes anônimos — policy restrita ao e-mail do admin (auth.jwt() ->> 'email' = 'santanadds92@gmail.com'); só fica visível para quem tiver sessão autenticada como admin.
Credenciais de admin (e-mail/senha)	CONFIGURADA no Supabase Auth	Fora do repositório — usuário criado manualmente no painel Supabase	O e-mail do admin, porém, está em texto plano dentro dos arquivos .sql do repositório (usado como condição das policies RLS). Isso não é uma senha exposta, mas é uma informação de identidade do administrador versionada em texto — vale considerar se o repositório GitHub é privado.

Arquitetura real de acesso: Navegador → **direto** → Supabase / Google Gemini API. Não existe Proxy/Serverless Function no meio. Isso é uma limitação estrutural conhecida de sites 100% estáticos sem backend — **não é um erro de implementação, é uma característica arquitetural** que precisa ser entendida como tal por qualquer parceiro técnico futuro: qualquer chave que precise ser lida por um visitante anônimo (como a do chat) **não pode, por definição, ficar 100% secreta** nessa arquitetura, a menos que se introduza uma camada de servidor (Vercel Serverless Function ou Supabase Edge Function agindo como proxy) — o que **não existe hoje neste projeto**.

[VERIFICADO NO PROJETO ATUAL] para todos os itens acima.

12. Proteção contra Vulnerabilidades

Vetor	Situação	Evidência
XSS armazenado (formulários públicos → admin)	Mitigado	escapeHTML() aplicado nas 3 funções de render do admin (Seção 5.1)

Vetor	Situação	Evidência
XSS via descrição de produto (rich text)	Mitigado	sanitizeDescricao() em supabase-client.js usa DOMPurify (com allowlist de tags) quando disponível; tem fallback de regex simples se DOMPurify não carregar — o fallback é mais fraco, mas DOMPurify está de fato carregado nas duas páginas onde a função é usada (admin.html, produto.html)
XSS no chat	Mitigado	linkify() em chatbot.js escapa &<> antes de converter URLs em links — texto do usuário nunca é injetado como HTML bruto
XSS via URL/query string (ex: ?codigo=, ?busca=)	[PARCIALMENTE VERIFICADO]	fetchProdutoByCodigo/fetchProdutos passam o parâmetro para o Supabase via método do SDK (.eq(), .ilike()), que parametriza a query — não há concatenação manual de SQL. Não foi auditado se o valor da query string é re-inserido em innerHTML em algum ponto de produto.html/produtos.html sem escape; não foi lido o JS dessas duas páginas nesta rodada
SQL Injection	Não aplicável na prática	Não há SQL bruto client-side; todo acesso passa pelo SDK oficial do Supabase (PostgREST), que parametriza chamadas
Command Injection	Não aplicável	Não há execução de comando de sistema em nenhuma camada deste projeto (site estático)
Prompt Injection (chatbot)	Parcialmente mitigado, não testado ativamente	O SYSTEM_INSTRUCTION é fixo e enviado separadamente do texto do usuário (campo systemInstruction, não concatenado à entrada) — isso reduz o risco clássico de injeção direta, mas não há nenhum filtro/guarda adicional contra tentativas de manipular a persona do modelo dentro da própria conversa. Nenhum teste de red-team foi executado

Vetor	Situação	Evidência
CSRF	Baixo risco estrutural	A autenticação do admin usa token JWT do Supabase Auth (Bearer, não cookie de sessão clássico dirigindo estado no servidor), o que reduz a superfície clássica de CSRF; não foi feita uma verificação formal
CORS	[NÃO VERIFICADO]	Configuração de CORS do projeto Supabase não é visível a partir do repositório (é config de painel, fora do código)
Autenticação/Autorização Exposição de endpoints	Ver Seções 11 e 25 Nenhum endpoint customizado existe (não há /api) — a única “API” é a REST automática do Supabase (protegida por RLS) e a API pública do Google Gemini (protegida por chave, não por este projeto)	— [VERIFICADO NO PROJETO ATUAL]
Validação de entrada	Campos obrigatórios via HTML5 (required, type="email", type="tel") — validação é só de front-end; o Postgres aplica not null em colunas-chave das tabelas (ex: nome, mensagem), mas não há validação de formato (regex de telefone/e-mail) no banco	[VERIFICADO NO PROJETO ATUAL]

13. Rate Limiting e Anti-Spam

[NÃO ENCONTRADO / NÃO APLICÁVEL — lacuna real, não um “não aplicável” arquitetural.]

Nenhum mecanismo de rate limiting, CAPTCHA, honeypot, verificação de IP/sessão ou cooldown foi encontrado em nenhum dos 4 formulários públicos (Fale Conosco, Bordado, Revenda, Checkout) nem no chat. Qualquer visitante anônimo pode, tecnicamente, enviar múltiplas submissões em sequência para mensagens_contato, solicitacoes_bordado ou solicitacoes_revenda sem nenhuma barreira própria do projeto.

O Supabase possui limites de taxa em nível de plataforma/projeto (aplicáveis a todo tráfego da API), mas isso é uma proteção de infraestrutura do provedor, não uma decisão de engenharia deste projeto, e seus parâmetros exatos não são visíveis a partir do repositório. **[NÃO VERIFICADO]**

Recomendação registrada, não implementada: um honeypot simples (campo invisível que só bots preenchem) ou um CAPTCHA leve (ex: Cloudflare Turnstile) nos 4 formulários públicos reduziria spam sem exigir backend próprio.

14. LGPD e Privacidade

Diferenciação explícita entre implementação técnica e conformidade jurídica, conforme exigido:

O que existe tecnicamente [VERIFICADO NO PROJETO ATUAL]:

- `politica-de-privacidade.html` e `termos-de-uso.html`, com razão social e CNPJ reais.
- Detalhamento de quais dados são coletados por formulário, finalidade, retenção, e os terceiros envolvidos (WhatsApp/Meta, Google Gemini, Supabase).
- Checkbox required de consentimento nos 4 formulários que coletam dado pessoal, com link para os dois documentos.
- Seção dedicada aos direitos do titular (Art. 18 da LGPD) e ao direito de arrependimento de 7 dias (Art. 49 CDC).
- Divulgação do uso de `localStorage` apenas para carrinho de compras (sem cookies de rastreamento de terceiros).

O que NÃO está confirmado/garantido:

- **Não há revisão jurídica confirmada** deste conteúdo por um advogado especializado em LGPD — o texto foi redigido com base em boa prática técnica geral, não é aconselhamento jurídico. **[NÃO VERIFICADO]**
- **Não há mecanismo técnico de exclusão automatizada de dados** (o “direito de eliminação” hoje depende de um pedido manual por e-mail, processado manualmente pelo admin — não existe um botão de “excluir meus dados” ou rotina automatizada) — a política promete o direito, mas o cumprimento é operacional/manual, não sistêmico.
- **Não há registro de consentimento versionado** (não se grava no banco *qual* versão da política o usuário aceitou, nem timestamp do aceite — só o fato de que o checkbox precisa ser marcado para o formulário ser enviado, o que é validação de front-end, contornável por quem manipular a requisição diretamente).
- **Retenção de dados não tem rotina de expiração automática** — as tabelas `mensagens_contato`, `solicitacoes_bordado`, `solicitacoes_revenda` não têm nenhum TTL/job de limpeza; os dados ficam indefinidamente até exclusão manual pelo admin.

Conclusão da seção: o projeto tem uma **base técnica de conformidade LGPD implementada corretamente para o nível de um e-commerce de pequeno/médio porte**, mas isso não equivale a uma certificação ou garantia jurídica de conformidade total — é uma implementação de boa-fé que reduz risco, não uma auditoria jurídica formal.

15. Banco de Dados — Supabase

Projeto: `aixudpelpjyuwpsocikk` (identificador de URL, não é secreto) **[VERIFICADO NO PROJETO ATUAL]**

Tabela produtos

Coluna	Tipo	Observações
<code>id</code> <code>codigo</code>	<code>uuid</code> , PK text, unique , not null	default <code>gen_random_uuid()</code> usado como chave de negócio (URL do produto usa <code>?codigo=</code>)
<code>titulo</code>	text, not null	

Coluna	Tipo	Observações
categoria	text, not null	valores esperados: vestuario, calcados, mochilas, insignias, protecao, facas, kits, acessorios, replicas (não há CHECK constraint — validação só na camada de aplicação)
corporacao	text	ex: EB, MB, FAB, PMESP, Geral
descricao	text	HTML sanitizado no client antes de exibir
tamanhos, cores	text[], default '{}'	arrays Postgres
preco, preco_promocional	numeric	promocional pode ser null
estoque_status	text, default 'disponivel'	disponivel / sob_encomenda / esgotado — sem CHECK
personalizavel	boolean, default false	
fotos	text[], default '{}'	URLs do bucket produtos-fotos
destaque	boolean, default false	
status	text, default 'ativo'	ativo / inativo / arquivado — sem CHECK
created_at	timestampz, default now()	

Tabela mensagens_contato

id (uuid PK), nome (not null), email, telefone, mensagem (not null), lida (boolean, default false), created_at.

Tabela solicitacoes_bordado

id (uuid PK), nome (not null), telefone (not null), tipo_peca, o_que_bordar, observacoes, status (default 'novo'), created_at.

Tabela solicitacoes_revenda

id (uuid PK), nome (not null), telefone (not null), cidade, tipo_negocio, observacoes, status (default 'novo'), created_at.

Tabela site_config

chave (text, PK), valor (text), updated_at (timestampz, default now()) — modelo chave/valor genérico, hoje usado só para as duas chaves de API do Gemini.

Índices explícitos: nenhum índice adicional além das primary keys/unique constraints padrão foi encontrado nos scripts SQL — não há índice dedicado em categoria, status ou codigo (além do unique que já cria um índice implícito). Para o volume atual (~38 produtos semeados), isso não é um problema; se o catálogo crescer para milhares de itens, faltaria indexação em categoria/status para as queries de listagem filtrada. **[VERIFICADO NO PROJETO ATUAL]**

Views, functions, triggers, sequences customizadas: **[NÃO ENCONTRADO]** — nenhum desses objetos foi definido nos scripts SQL do projeto; toda a lógica de negócio vive no JavaScript do frontend.

16. Row Level Security (RLS)

Tabela	RLS ativo?	Policy	Operação	Papel/condição
produtos	Sim	“Public read produtos ativos”	SELECT	qualquer um, só onde status='ativo'
produtos	Sim	“Admin full access produtos”	ALL	só auth.jwt()->>'email' = 'san-tanadds92@gmail.com'
mensagens_contato	Sim	“Anon insert mensagens_contato”	INSERT	qualquer um (with check (true))
mensagens_contato	Sim	“Admin manage/update/delete mensagens_contato”	SELECT/UPDATE/DELETE	só admin
solicitacoes_bordado	Sim	mesmo padrão de mensagens_contato	INSERT público / SELECT-UPDATE-DELETE admin	—
solicitacoes_revenda	Sim	mesmo padrão	INSERT público / SELECT-UPDATE-DELETE admin	—
site_config (pós-hardening)	Sim	“Public read chatbot key only”	SELECT	só onde chave='chatbot_gemini_key'
site_config	Sim	“Admin read site_config”	SELECT	só admin (todas as chaves)
site_config	Sim	“Admin insert/update site_config”	INSERT/UPDATE	só admin
storage.objects (bucket produtos-fotos)	Sim (política de Storage)	“Public read produtos-fotos”	SELECT	qualquer um
storage.objects	Sim	“Admin upload/delete produtos-fotos”	INSERT/DELETE	só admin

Avaliação de risco das policies: - Nenhuma tabela do projeto está **sem RLS** — as 4 tabelas de negócio + site_config têm RLS habilitado explicitamente (alter table ... enable row level security). **[VERIFICADO NO PROJETO ATUAL]** - O padrão “INSERT liberado para anon, leitura/gestão só para admin” nas 3 tabelas de lead (mensagens/bordado/revenda) é apropriado para formulários públicos — é exatamente o necessário e nada mais. - A policy de leitura de site_config pré-hardening (“Public read site_config”, liberando a tabela inteira) foi **identificada como excessivamente permissiva e corrigida** por supabase-admin-gemini-key.sql, que a substitui por uma policy restrita só à linha do chat. **Ressalva:** essa substituição só tem efeito se o script supabase-admin-gemini-key.sql de fato foi executado no projeto Supabase real — isso não pode ser confirmado a partir do repositório (é uma ação de painel, fora do controle de versão). **[NÃO VERIFICADO]** se a migration foi aplicada no ambiente de produção. - Autorização de admin é feita 100% por comparação direta de e-mail (auth.jwt()->>'email' = '...') em cada policy, e não por uma tabela de roles/claims customizada. Isso funciona para um single-admin, mas não escala bem se a loja precisar de múltiplos administradores com permissões diferentes no futuro — mudar o e-mail do admin exigiria editar e re-rodar todas as policies em todos os scripts SQL. - Acesso anônimo (anon) está corretamente limitado a INSERT-only nas tabelas de lead, e a SELECT-only (produtos ativos) na vitrine — nenhuma policy libera UPDATE/DELETE para anon em nenhuma tabela.

17. APIs, Edge Functions e Serverless

[NÃO APLICÁVEL A ESTE PROJETO] — não existe nenhuma Vercel Serverless Function, Vercel Edge Function, ou Supabase Edge Function no repositório (não há pasta `/api`, nem `supabase/functions/`). Toda “chamada de API” observada no projeto é:

1. **Frontend → Supabase REST (PostgREST)**, via SDK oficial, protegida por RLS — não é uma função customizada, é a API automática gerada pelo próprio Supabase a partir do schema do banco.
2. **Frontend → Google Generative Language API**, via `fetch()` direto do navegador (`assets/chatbot.js`, `assets/gemini-ai.js`) — sem nenhum proxy no meio.

O fluxo completo, portanto, é sempre **Navegador → Serviço externo**, nunca **Navegador → API própria → Backend**. Essa é a limitação arquitetural central deste projeto e a razão pela qual as chaves de API não podem ser mantidas 100% secretas (ver Seção 11).

18. Inteligência Artificial

Duas integrações **[VERIFICADO NO PROJETO ATUAL]**:

18.1 Autofill de produto (admin)

- **Modelo:** cascata `gemini-flash-latest` → `gemini-2.5-flash` → `gemini-3.6-flash` (o último nome é suspeito de imprecisão, ver Seção 4).
- **Arquitetura:** chamada direta do navegador (`admin.js` → `gemini-ai.js` → `fetch()` para `generativelanguage.googleapis.com`), sem proxy/Edge Function.
- **Prompt:** `buildProdutoPrompt()` em `gemini-ai.js` — prompt extenso e estruturado, exige JSON puro como saída (`responseMimeType: 'application/json'`), define categoria fechada (enum de 9 valores), exige HTML restrito a 5 tags (`p`, `h3`, `strong`, `ul`, `li`, `hr` — mais `hr` — na verdade a instrução lista `<p>`, `<h3>`, `<strong>`, `<ul>`, `<li>`, `<hr>`) e proíbe explicitamente inventar especificações técnicas não informadas pelo usuário.
- **Sanitização de saída:** o HTML retornado pela IA passa por `sanitizeDescricao()` (`DOMPurify`) antes de ser inserido no editor do admin — proteção correta contra a IA (ou um prompt malicioso) gerar HTML perigoso.
- **Fallback/erro:** cascata de modelos com `try/catch` por modelo, mensagens de erro específicas exibidas via toast no admin.
- **Rate limit:** nenhum implementado pelo projeto — depende só da cota da própria chave no Google Cloud Console.
- **Segurança:** chave restrita por RLS ao e-mail do admin (não exposta a visitante anônimo).

18.2 Chat público “Recruta QRV”

- **Modelo:** cascata `gemini-flash-latest` → `gemini-2.5-flash` → `gemini-1.5-flash`.
- **Arquitetura:** idêntica à anterior, direto do navegador.
- **System prompt:** fixo, define persona (“recruta” de caserna, fala em tom militar caricato), restringe a base de conhecimento a fatos literalmente listados no prompt (endereço, horário, frete, parcelamento, contato), e instrui a IA a **não inventar** dado que não esteja na lista, direcionando para o WhatsApp quando não souber.
- **Direcionamento comercial:** o prompt instrui explicitamente a IA a encaminhar para o WhatsApp humano em caso de fechamento de compra ou orçamento de bordado — o chat **não fecha vendas sozinho**, é um funil de atendimento, não um agente transacional.

- **Contexto/histórico:** mantido em variável `history` em memória do navegador (array de mensagens), **não persistido** em nenhum banco — se o visitante recarregar a página, perde o histórico da conversa.
- **Sanitização de entrada do usuário exibida na UI:** `linkify()` escapa `&<>` antes de renderizar (Seção 12).
- **Fallback:** se a chave ainda não carregou (`GEMINI_API_KEY` vazia), o widget tenta buscar de novo antes de desistir e mostrar mensagem de erro direcionando ao WhatsApp — bom tratamento de corrida entre carregamento assíncrono da chave e primeira mensagem do usuário.
- **Segurança:** chave exposta a qualquer visitante que inspecionar a rede (ver Seção 11) — mitigação é só operacional (Referrer restriction + cota).
- **Proteção contra prompt injection:** parcial, não testada ativamente (ver Seção 12).

Nenhuma chave de API real foi exibida em nenhum ponto deste dossiê, conforme exigido.

19. UI/UX e Componentização

- **Header/Footer:** unificados em `assets/components.js` — única fonte da verdade (Seção 5.3). Sem duplicação restante.
- **Hero:** implementação própria por página (só existe na Home), 3 slides com imagem dedicada desktop/mobile.
- **Navegação:** menu horizontal (desktop) + drawer lateral (mobile), ambos vindos do mesmo componente.
- **Vitrines/Produtos:** `productCardHTML()` em `supabase-client.js` é o **template único e reutilizado** de card de produto — usado tanto na Home (destaques) quanto em `produtos.html` (grade completa) e na seção de relacionados de `produto.html`, evitando duplicação de markup de card em múltiplos arquivos.
- **Formulários:** 4 formulários públicos + formulário de produto no admin, todos com padrão visual comum (`.search-panel`, `.form-row`, `.field`).
- **CTAs:** botão principal padronizado como *pill button* (44px/border-radius:25px) após rodada de ajuste fino.
- **Modais/Drawers:** drawer mobile (menu), lightbox (galeria de produto) — ambos implementação própria.
- **Chat:** widget próprio, componentizado dentro do próprio `chatbot.js` (a UI é montada via `buildWidgetHTML()`, não duplicada em HTML estático).
- **Painel administrativo:** único arquivo `admin.html` + `admin.js`, com sistema de abas internas (Produtos, Bordados, Revenda, Mensagens) — não são páginas separadas, são painéis (`panelList`, `panelBordados` etc.) alternados via JS.

Estados/eventos/listeners: delegação de evento global para `.add-to-cart-btn` (`document.addEventListener(...)` em `cart.js`) é um padrão sólido — funciona mesmo para cards de produto renderizados dinamicamente depois do carregamento inicial da página, sem precisar re-anexar listener por card.

Duplicações remanescentes identificadas: nenhuma duplicação estrutural de HTML entre páginas além da já eliminada (header/footer). A duplicação que resta é de **padrão de código**, não de markup: cada página com formulário (`contato.html`, `bordados.html`, `revendedor.html`) repete um bloco `<script type="module">` quase idêntico de `addEventListener('submit', ...)` com `try/catch/toast` — funcional, mas poderia ser extraído para uma função utilitária compartilhada em `supabase-client.js`. **[VERIFICADO NO PROJETO ATUAL]**, classificado como oportunidade de refatoração, não como bug.

20. Identidade Visual e Efeitos Especiais

[PARCIALMENTE VERIFICADO] — o CSS completo (1208 linhas) não foi lido linha a linha nesta rodada; a análise abaixo é baseada nas classes e efeitos confirmados durante a inspeção de

header/footer/hero/CLS:

- **Paleta:** tons dourado/gold (--gold-light, classes .gold-rule, .btn-gold) sobre base escura — confirmado nos textos legais e nos botões (btn-gold, btn-outline).
- **Tipografia:** três famílias (Inter, Oswald, Poppins) com pesos de 300 a 900 — indica hierarquia visual deliberada (provavelmente Oswald para títulos de impacto militar, Inter/Poppins para corpo de texto — não confirmado no CSS nesta rodada, **[NÃO VERIFICADO]** o mapeamento exato).
- **Efeitos confirmados:** content-visibility/contain-intrinsic-size (performance, não visual); sticky positioning em sidebars/galeria/resumo de carrinho.
- **Canvas/partículas/parallax:** **[NÃO ENCONTRADO]** — nenhuma referência a <canvas>, biblioteca de partículas ou efeito de parallax foi localizada no projeto.
- **Ícones:** SVG inline (não sprite sheet, não biblioteca de ícones como Font Awesome/Lucide) — cada ícone (Instagram, WhatsApp, busca, carrinho, conta, envelope) é um <svg> escrito à mão diretamente no HTML/components.js.

21. Mídia e Assets Responsivos

Tipo	Quantidade aproximada	Observação
.png	maioria dos banners/hero/logo/selos/favicons	Nenhuma variante
.jpg	~38 fotos de catálogo (assets/produtos-catalogo/) + 3 fotos de exemplo de produto na raiz de assets/	
.svg	favicon	ícones de UI são SVG inline , não arquivo .svg separado
.ico	favicon	
Vídeo (.mp4/.webm)	[NÃO ENCONTRADO]	Nenhum arquivo de vídeo no projeto
Fontes	via Google Fonts CDN (não hospedadas localmente)	

srcset/<picture>: **[NÃO ENCONTRADO]** — nenhuma imagem usa variantes de resolução via srcset; a adaptação mobile é feita trocando o arquivo de imagem inteiro por breakpoint via CSS (background-image diferente), não por <picture>/srcset no HTML. **poster/autoplay/muted/playsinline:** **[NÃO APLICÁVEL]** — sem vídeo, não há o que configurar. **Lazy loading:** aplicado seletivamente (Seção 6), não de forma universal a todas as do site.

22. Acessibilidade — WCAG

Nenhuma auditoria formal (axe-core, Lighthouse Accessibility, WAVE) foi executada neste engajamento. Os achados abaixo são de **inspeção manual de código**, não uma auditoria completa — portanto a lista é necessariamente incompleta.

Achado	Severidade	Evidência
aria-label presente em botões de ícone (menu, busca, carrinho, conta, fechar drawer/chat, WhatsApp)	Positivo	components.js, chatbot.js
alt presente nas imagens inspecionadas (logo, selos de pagamento, imagens de produto no card)	Positivo	productCardHTML(), components.js
Falta de teste de navegação 100% por teclado (Tab, Enter, Esc para fechar modais)	Médio	Não testado — comportamento de foco em drawer/lightbox/chat não confirmado
Falta de auditoria de contraste de cor (paleta dourado sobre escuro pode ter zonas de contraste insuficiente, especialmente em texto secundário/text-muted)	Médio	Não medido com ferramenta (ex: axe, contrast checker)
Uso de <h1>--<h4> — hierarquia de headings não auditada em todas as 15 páginas nesta rodada	Baixo/Médio	[NÃO VERIFICADO]
<label> associado a inputs de formulário — confirmado no padrão .field (label + input adjacentes), mas sem confirmar for/id explícito em 100% dos casos	Médio	[PARCIALMENTE VERIFICADO]
Landmarks ARIA (role="main", role="navigation" explícitos)	[NÃO VERIFICADO]	HTML5 semântico (<header>, <nav>, <footer>) está presente, o que já supre parte da necessidade de landmarks mesmo sem role explícito
Vídeos com legenda/transcrição	[NÃO APLICÁVEL]	Sem vídeo no site
Carrossel do blog navegável por teclado	[NÃO VERIFICADO]	Botões #blogPrev/#blogNext são <button> nativos (focáveis por padrão), mas não foi testado se Enter/Space disparam corretamente em uso real

Conclusão: não é possível declarar conformidade WCAG (nível A, AA ou AAA) para este projeto — o que existe são boas práticas pontuais (uso de HTML semântico, aria-label em ícones, alt em imagens), não uma certificação. Recomenda-se rodar uma auditoria automatizada (Lighthouse/axe) no ambiente publicado como próximo passo.

23. SEO Técnico

Diagnóstico por elemento (aplicável a todas as 14 páginas públicas, salvo indicação contrária):

Elemento	Status	Detalhe
<title>	[OK] Presente e único por página	Confirmado em index.html, termos-de-uso.html, politica-de-privacidade.html, revendedor.html, contato.html
<meta name="description">	[OK] Presente em todas as páginas	Adicionado como parte da rodada de SEO da auditoria original
<link rel="canonical">	[FALTA] Ausente em todas as páginas	Nenhuma ocorrência encontrada — risco de conteúdo duplicado não sinalizado (ex: mesma página acessível com/sem parâmetros de query)
robots.txt (arquivo na raiz)	[FALTA] Não encontrado	Nenhum arquivo robots.txt no repositório
sitemap.xml	[FALTA] Não encontrado	Nenhum sitemap gerado
Open Graph (og:title, og:description, og:image etc.)	[FALTA] Ausente em todas as páginas	Compartilhamento em redes sociais (WhatsApp, Instagram, Facebook) não terá preview rico — impacto direto, já que WhatsApp é o canal de venda principal do negócio
Twitter Cards schema.org/dados estruturados (application/ld+json)	[FALTA] Ausente [FALTA] Ausente em todas as páginas	— Nenhum Product, Organization, BreadcrumbList ou LocalBusiness schema — oportunidade perdida para rich snippets no Google, especialmente relevante para uma loja com endereço físico e catálogo de produtos
meta name="robots"	Presente só em 4 páginas	admin.html/checkout.html = noindex, nofollow (correto — não devem ser indexadas); politica-de-privacidade.html/termos-de-uso.html = noindex, follow (correto — conteúdo legal não precisa competir por ranking, mas passa autoridade de link). As demais 10 páginas públicas não têm a tag, o que é equivalente ao padrão index, follow (correto por omissão)
Favicon	[OK] Completo	.svg, múltiplos .png (16/32/192/512), .ico, apple-touch-icon
manifest.json/manifest.webmanifest (PWA)	[FALTA] Não encontrado	Site não é instalável como PWA

Elemento	Status	Detalhe
URLs amigáveis	Parcial	Páginas com nome descritivo (produtos.html, contato.html), mas produto individual usa query string (produto.html?codigo=QRV-INS-001) em vez de URL amigável (/produto/QRV-INS-001) — limitação normal de site estático sem roteamento de servidor
Imagens com alt	Presente nos pontos auditados (Seção 22)	Não auditado 100%
Redirects/404 customizado	[NÃO VERIFICADO]	Depende de configuração da Vercel, não visível no repositório

Conclusão da seção: esta é a área com a lacuna técnica mais concreta e concentrada do projeto. Meta description e favicon estão bem resolvidos, mas a ausência total de Open Graph, dados estruturados, sitemap e canonical é uma limitação real que afeta tanto o SEO orgânico quanto — de forma mais imediata para este negócio específico — a aparência dos links compartilhados no WhatsApp (que é o canal comercial primário da loja).

24. Dependências e Supply Chain

Dependência	Origem	Versão fixada?	Risco	Pode ser removida?
@supabase/supabase-js@2	CDN esm.sh	Major fixado (@2), minor/patch flutuante	Médio — mudanças de minor version podem alterar comportamento sem aviso; disponibilidade depende do CDN esm.sh estar no ar	Não — é o core de acesso a dados
DOMPurify @3	CDN cdn.jsdelivr.net	Major fixado (@3), minor/patch flutuante	Baixo-médio — mesma lógica de versão flutuante; é uma biblioteca de segurança ativa e bem mantida	Não — é a principal defesa contra XSS de conteúdo rico
Google Fonts (Inter, Oswald, Poppins)	fonts.googleapis.com	N/A (serviço, não pacote)	Baixo	Sim, tecnicamente (self-host das fontes eliminaria a requisição externa), mas não é urgente

Dependência	Origem	Versão fixada?	Risco	Pode ser removida?
Google Generative Language API	generativelanguage.googleapis.com	Modelos com fallback em cascata (não fixados a uma versão exata)	Médio-alto — depende de disponibilidade contínua de pelo menos um dos 3 modelos candidatos; Google já descontinuou modelos Gemini no passado (mencionado no próprio comentário do código)	Não sem redesenhar o chat/autofill
WhatsApp (wa.me)	Link direto, sem SDK	N/A	Baixo	Não — é o canal de conversão principal

Nenhum gerenciador de pacotes (npm/yarn/pnpm) é usado — não há package-lock.json para auditar (npm audit não se aplica a este projeto). Todo o “supply chain” do projeto é: (1) as duas dependências via CDN acima, fixadas por major version, e (2) as APIs externas de terceiros (Google, Supabase, WhatsApp). Não foram encontradas dependências duplicadas, obsoletas com CVE conhecida, ou trackers/analytics de terceiros (nenhum Google Analytics, Meta Pixel, Hotjar ou similar foi localizado no projeto). **[VERIFICADO NO PROJETO ATUAL]**

25. Autenticação e Painel Administrativo

- **Login:** e-mail/senha via `supabase.auth.signInWithPassword()` (admin.js linha 235). **[VERIFICADO NO PROJETO ATUAL]**
- **Sessão:** verificada com `supabase.auth.getSession()` no carregamento (admin.js linha 192) — decide se mostra a tela de login ou o painel.
- **Logout:** `supabase.auth.signOut()` (linha 245), acionado pelo botão #logoutBtn.
- **Roles/permisões:** não há sistema de roles — existe **um único administrador**, identificado por e-mail fixo (santanadds92@gmail.com) hardcoded em todas as policies RLS relevantes (produtos, mensagens, bordado, revenda, site_config, storage). Não há conceito de “admin secundário” ou nível de permissão diferenciado.
- **Rotas protegidas:** não há roteamento server-side — a “proteção” de `admin.html` é inteiramente client-side (a página carrega, mas os dados só aparecem/funcionam se a sessão Supabase for válida; a proteção real e efetiva é o RLS do banco, não o carregamento da página em si). Isso significa que **qualquer pessoa pode abrir a URL admin.html** e ver a interface de login — o que é normal e não é uma falha —, mas não conseguirá ler/escrever nenhum dado sem autenticar, porque isso é bloqueado no banco, não apenas escondido na tela.
- **CRUD:** completo para produtos (criar, listar, editar, excluir) e para gestão de leads (listar/atualizar status/excluir mensagens, bordados, revendas).
- **Recuperação de senha:** **[NÃO VERIFICADO]** — não foi localizado nenhum fluxo de “esqueci minha senha” customizado no `admin.html`; se existir, seria o fluxo padrão do Supabase Auth (fora do controle deste código).
- **Possibilidade de acesso administrativo indevido:** o único vetor plausível seria comprometer a conta de e-mail/senha do admin (fora do escopo de código) ou uma falha de RLS — as

polítics foram revisadas e não apresentam brecha óbvia (nenhuma policy libera escrita para anon em tabelas sensíveis). **[VERIFICADO NO PROJETO ATUAL]**

26. Tratamento de Erros e Observabilidade

- **try/catch:** presente de forma consistente em todas as chamadas assíncronas relevantes — submissão de formulários (contato.html, bordados.html, revendedor.html), chamadas Gemini (callGeminiChat, callGemini), operações do admin (login, CRUD, upload de fotos, geração por IA).
- **Feedback ao usuário:** sistema de “toast” (showToast() no admin, cart-mini-toast no site público) para sucesso/erro — não são apenas alert() (exceto em 2 pontos observados: contato.html/revendedor.html usam alert('Erro ao enviar: ' + err.message) diretamente no catch do submit, em vez de um toast estilizado — inconsistência menor de UX de erro).
- **Fallback de modelo de IA:** cascata de até 3 modelos Gemini com mensagens de erro específicas por falha (Seção 18).
- **Logs:** apenas console.error() no navegador do próprio usuário/admin — **não há nenhum serviço de logging/monitoramento centralizado** (Sentry, LogRocket, Vercel Analytics, ou equivalente) integrado ao projeto.
- **Analytics: [NÃO ENCONTRADO]** — nenhum Google Analytics, Meta Pixel, Vercel Analytics ou similar foi localizado em nenhuma página.
- **404/500:** páginas de erro customizadas não foram encontradas no repositório — comportamento de 404 depende do padrão da Vercel para sites estáticos. **[NÃO VERIFICADO]**
- **Pontos únicos de falha (SPOF) identificados:**
 1. **assets/supabase-client.js** — se o import do CDN esm.sh falhar (rede/CDN fora do ar), absolutamente nenhuma função dinâmica do site funciona (catálogo, formulários, carrinho de checkout com dados reais, login admin).
 2. **A chave Gemini do chat** — se a policy RLS de site_config ou o valor da chave estiverem incorretos, o chat falha silenciosamente para “chave não configurada”, mas o restante do site continua funcionando (bom isolamento de falha nesse caso específico).
 3. **components.js** — se este script falhar ao carregar (erro de rede, bloqueio de CDN corporativo etc.), **toda página perde header e footer inteiros**, incluindo a navegação principal — é o SPOF mais crítico de UI do projeto, por ser a fonte única de header/footer em todas as 14 páginas.

27. Deploy e CI/CD

Não há pipeline de CI/CD tradicional (GitHub Actions, testes automatizados, lint automático) — o “CI/CD” deste projeto é o deploy automático nativo da Vercel a cada push na branch main. **[VALIDADO EM AUDITORIA/HISTÓRICO]**

Comandos reais e verificados no repositório:

Instalação/desenvolvimento local — não há passo de instalação (npm install) porque não há dependências de pacote; para rodar localmente basta servir os arquivos estáticos (ex: npx serve . ou a extensão Live Server, não documentado formalmente no projeto).

Deploy (push.bat, Windows — conteúdo real do arquivo):

```
git add -A
git commit -m "%MSG%"
git push origin main
```

Deploy (deploy.sh, macOS/Linux/Git Bash — conteúdo real do arquivo):

```
git add -A
git diff --cached --quiet || { git commit -m "$MSG"; git push origin main; }
```

Rollback: não há script de rollback automatizado no projeto. O procedimento real seria `git revert <hash-do-commit>` seguido de `git push origin main` (a Vercel republicaria automaticamente a versão revertida), ou reverter manualmente pelo painel da Vercel para um deploy anterior (recurso nativo da plataforma, fora do repositório). **[NÃO VERIFICADO diretamente — inferido do fluxo padrão Git/Vercel, não testado nesta sessão]**

Migrations do banco: não há sistema de migration versionado (tipo Supabase CLI migrations/) — os 4 arquivos `.sql` do projeto são scripts avulsos, pensados para serem colados manualmente no SQL Editor do painel Supabase, **na ordem**: `supabase-setup.sql` → `supabase-site-config.sql` → `supabase-admin-gemini-key.sql` → `seed-produtos-catalogo.sql` (ordem inferida da lógica de dependência entre eles, ex: `supabase-admin-gemini-key.sql` pressupõe que `site_config` já existe). **[VERIFICADO NO PROJETO ATUAL]** quanto ao conteúdo; a ordem de execução real no ambiente de produção não é verificável a partir do repositório.

Branches/preview: a Vercel, por padrão, gera deploy de preview para qualquer branch/PR além da main — mas isso é comportamento padrão da plataforma, não uma configuração customizada encontrada no repositório (não há `vercel.json` restringindo isso). **[NÃO VERIFICADO]**

28. Variáveis de Ambiente

Variável	Ambiente	Finalidade	Público/Privado	Onde é usada
—	—	—	—	—

[NÃO ENCONTRADO / NÃO APLICÁVEL] — este projeto **não usa arquivo `.env/.env.local` nem variáveis de ambiente da Vercel**. Todas as configurações que normalmente estariam em variáveis de ambiente (URL e chave pública do Supabase) estão hardcoded diretamente em `assets/supabase-client.js`, porque são valores destinados a serem públicos (a chave é a “publishable/anon key”, protegida por RLS, não um secret). As chaves que de fato precisam de sigilo (Gemini API keys) **não estão em variável de ambiente nem no código-fonte** — estão armazenadas na tabela `site_config` do próprio banco Supabase, geridas via UI do admin, não via deploy. Essa é uma escolha de arquitetura deliberada e coerente com “zero backend/zero build step” (Seção 5.4/5.5), mas significa que o projeto **não tem, hoje, nenhum uso do sistema de variáveis de ambiente da Vercel**.

29. Backup e Disaster Recovery

Backup: - **Código:** versionado em Git/GitHub — histórico completo de commits disponível; é o backup natural do frontend. - **Banco de dados:** backup é responsabilidade do plano Supabase contratado (Supabase oferece backups automáticos diários em planos pagos, point-in-time recovery em planos superiores) — **não verificável a partir deste repositório**, depende do plano ativo no painel Supabase. **[NÃO VERIFICADO]** - **Assets/mídia do catálogo:** as fotos enviadas via admin ficam no bucket `produtos-fotos` do Supabase Storage — sujeitas à mesma política de backup do projeto Supabase, não têm uma cópia adicional fora dele conhecida por este repositório. - **Domínio/DNS:** fora do escopo deste repositório — gerido onde o domínio foi registrado (não identificado no código). - **Secrets:** as chaves Gemini vivem só no banco (`site_config`); não há um cofre de secrets separado (ex: `1Password`, `Vercel env`) — perder acesso ao Supabase (sem backup de banco) significa perder essas chaves também, exigindo recadastro manual pelo admin.

Procedimento de reconstrução do zero (adaptado à arquitetura real deste projeto)

1. **Preparar ambiente:** nenhuma instalação de runtime é necessária além de um editor de código e um navegador (não há Node.js obrigatório para rodar o site, só para servir localmente se

desejado).

2. **Clonar o repositório:** `git clone <url-do-repositório>` (a partir do GitHub onde o projeto está hospedado).
3. **Servir localmente (opcional, para testes):** qualquer servidor estático simples (ex: `npx serve .`, extensão “Live Server” do VS Code) — **não** abrir os arquivos direto via `file://`, porque isso pode gerar comportamento diferente do ambiente publicado em alguns navegadores/extensões.
4. **Criar um novo projeto Supabase** (ou restaurar o existente, se disponível): em `supabase.com`, criar projeto novo, anotar a URL e a chave `anon/publishable`.
5. **Rodar os scripts SQL, nesta ordem, no SQL Editor do Supabase:**
 - `supabase-setup.sql` (cria as 4 tabelas centrais + RLS + bucket de Storage) — **antes de rodar, substituir o e-mail `santanadds92@gmail.com` pelo e-mail real do admin em todas as ocorrências**, conforme instrução no próprio arquivo.
 - `supabase-site-config.sql` (cria `site_config`).
 - `supabase-admin-gemini-key.sql` (endurece a policy de `site_config` e cria a linha `produtos_gemini_key`).
 - `seed-produtos-catalogo.sql` (opcional — popula o catálogo inicial com os ~38 produtos já cadastrados).
6. **Criar o usuário admin:** em Authentication → Users no painel Supabase, criar manualmente um usuário com o mesmo e-mail usado nas policies do passo 5.
7. **Atualizar assets/supabase-client.js:** substituir `SUPABASE_URL` e `SUPABASE_ANON_KEY` pelos valores do novo projeto Supabase (Project Settings → API).
8. **Configurar as chaves de IA:** logar no `admin.html` com o usuário criado no passo 6, colar as chaves do Gemini nos dois campos correspondentes (chat e autofill de produto) — nada disso é feito por variável de ambiente, é uma ação de UI pós-deploy.
9. **Publicar na Vercel:** conectar o repositório GitHub a um novo projeto Vercel (importação padrão de site estático, sem necessidade de configurar build command, já que não há build step) — ou usar `vercel --prod` via CLI, se preferir.
10. **Configurar domínio (se aplicável):** apontar o domínio customizado no painel da Vercel, seguindo as instruções de DNS que a própria plataforma fornece.
11. **Testar:** repetir o checklist da Seção 33 (drawer mobile, carrinho, formulários com consentimento, chat, painel admin) no ambiente recém-publicado.
12. **Publicações seguintes:** usar `push.bat/deploy.sh` para automatizar `git add → commit → push`, disparando o redeploy automático da Vercel.

30. Testes

Área	Resultado	Evidência/Status
Funcional (CRUD admin, formulários, carrinho)	Verificado por leitura de código	Sem execução end-to-end automatizada; sem framework de teste (Jest, Playwright, Cypress) no projeto
Desktop (visual)	Verificado parcialmente	Ajustes CSS validados por balanço de chaves <code>{}</code> ; sem captura de tela/comparação visual real nesta sessão
Mobile (visual)	Verificado parcialmente	Mesma limitação — sem viewport real testado
Responsivo (breakpoints) API (Supabase)	Não testado ao vivo Verificado por leitura de código	[NÃO VERIFICADO] Chamadas usam SDK oficial; sem teste de integração automatizado

Área	Resultado	Evidência/Status
Banco de dados (RLS)	Verificado por leitura de policy	Sem teste automatizado de tentativa de acesso indevido (ex: script simulando um anon tentando UPDATE em produtos)
Segurança (XSS)	Verificado por revisão de código	Sem varredura automatizada (ex: OWASP ZAP) executada
Acessibilidade	Não auditado formalmente	[NÃO VERIFICADO] — ver Seção 22
SEO	Auditado manualmente	Gaps documentados na Seção 23
Performance	Não medido	[NÃO VERIFICADO] — nenhum Lighthouse/PageSpeed rodado
Cross-browser	Não testado	[NÃO VERIFICADO] — nenhuma verificação em Firefox/Safari/Edge realizada nesta sessão
Sintaxe JS	[OK] Passou	node --check usado em rodadas anteriores para validar sintaxe dos arquivos JS editados
Integridade de tags HTML	[OK] Passou	Contagem de <form>/</form>, <div id="site-header"> etc. balanceada, verificada em múltiplas rodadas

Conclusão honesta: os testes realizados neste engajamento foram, em sua esmagadora maioria, **verificação estática de código** (leitura, contagem, grep, checagem de sintaxe) — não testes dinâmicos em ambiente real de execução (navegador ao vivo, dispositivo físico, ferramenta de auditoria automatizada). Isso é uma limitação real e reconhecida deste processo, não um teste “feito e aprovado” que deveria ser lido como equivalente a QA completo.

31. Matriz de Riscos

Risco	Severidade	Probabilidade	Impacto	Mitigação	Status
Chave Gemini do chat exposta no navegador	Média	Alta (é visível a qualquer visitante técnico)	Uso indevido da cota/custo da API	Restrição de HTTP Referrer + cota diária no Google Cloud Console (documentado, mas não confirmável se foi de fato configurado)	Mitigado operacionalmente, não eliminado — [NÃO VERIFICADO] se a restrição foi ativada no console do Google

Risco	Severidade	Probabilidade	Impacto	Mitigação	Status
Ausência de rate limiting/anti-spam nos formulários	Média	Média	Poluição da caixa de leads, possível abuso de custo indireto (cada submissão aciona insert no banco)	Nenhuma implementada	Em aberto
site_config com policy pública se supabase-admin-gemini-key.sql não tiver sido rodado no ambiente real	Alta (se não aplicado)	Desconhecida	Chave de autofill de produto ficaria legível por qualquer visitante	Script existe e está correto no repositório	[NÃO VERIFICADO] — depende de confirmação no painel Supabase real
Falha do CDN (esm.sh ou jsdelivr)	Alta (se ocorrer)	Baixa	Site inteiro perde funcionalidade dinâmica (Supabase) ou sanitização (DOMPurify)	Nenhum fallback/self-host configurado	Em aberto
components.js falhar ao carregar	Alta	Baixa	Todas as páginas perdem header/footer/navegação	Nenhum fallback	Em aberto
Ausência de Open Graph/schema.org	Baixa-Média	Certa (já é a realidade)	Compartilhamento no WhatsApp/redes sociais sem preview rico; perda de oportunidade de rich snippet no Google	Nenhuma	Em aberto
Modelo gemini-3.6-flash inválido na cascata do admin	Baixa	Baixa (é só o 3º fallback)	Nenhum impacto prático hoje (primeiro modelo da lista funciona)	Nenhuma	Cosmético, recomendado corrigir
Retenção indefinida de dados pessoais sem rotina de expiração	Média	Certa (é a realidade atual)	Risco de compliance LGPD de longo prazo	Nenhuma automatizada — processo manual	Em aberto

Risco	Severidade	Probabilidade	Impacto	Mitigação	Status
Único admin, autorização hardcoded por e-mail em SQL	Baixa-Média	Baixa	Dificuldade operacional de trocar/adicionar admin; e-mail do admin versionado em texto no repositório	Nenhuma	Aceitável para o porte atual, merece nota para o futuro
Ausência de teste de carga/performance real	Média	Baixa (tráfego atual provavelmente baixo/médio)	Core Web Vitals reais desconhecidos, podem não corresponder às boas práticas de código aplicadas	Nenhuma	Recomenda-se medir assim que publicado
Falta de backup próprio do catálogo fora do Supabase	Média	Baixa	Se o projeto Supabase for perdido/excluído sem backup do provedor, perde-se o catálogo dinâmico (o seed-produtos-catalogo.sql serve como uma cópia parcial de recuperação)	seed-produtos-catalogo.sql funciona como backup parcial do catálogo inicial	Parcialmente mitigado

32. Matriz Final de Qualidade

Categoria	Nota Desktop	Nota Mobile	Evidência/Justificativa
Visual/UI	9	8,5	Consistente após componentização; várias rodadas de refinamento documentadas
UX	8,5	8,5	Fluxo de compra funcional via WhatsApp; falta checkout de pagamento integrado nativamente

Categoria	Nota Desktop	Nota Mobile	Evidência/Justificativa
Responsividade	8,5	8,5	Adaptações reais no CSS, mas não testadas em viewport/dispositivo ao vivo
Performance	8	7,5	Boas técnicas de código aplicadas, mas zero métrica real medida — nota reflete essa incerteza
Segurança	8,5	8,5	XSS mitigado, RLS bem estruturado; limitação estrutural de chave de IA exposta e ausência de rate limiting
Funcionalidade	9	9	pesam a nota CRUD completo, carrinho, chat, formulários — tudo funcional em código
Estabilidade visual (CLS)	7,5	7,5	Mitigação parcial (content-visibility), sem aspect-ratio generalizado nem medição real
SEO técnico	6	6	Meta description ok; ausência total de OG/schema/sitemap/canonical
Acessibilidade	6,5	6,5	pesa consideravelmente Boas práticas pontuais, sem auditoria formal
Qualidade geral	8,3	8,1	Média ponderada informal das categorias acima, com peso maior para Segurança/Funcionalidade e menor para itens não medidos ao vivo

Estas notas são **deliberadamente mais conservadoras** do que os vereditos de 9,5/9,5 registrados em rodadas anteriores desta mesma conversa — porque este dossiê pondera explicitamente categorias (SEO técnico, acessibilidade, performance medida, rate limiting) que não tinham peso equivalente nas avaliações anteriores, e a instrução para este documento foi explícita em não presumir nota máxima sem evidência de teste real.

33. Checklist Final de Aceite

- ☒ Arquitetura documentada e coerente com o código real (sem backend próprio; Supabase + Vercel + Gemini direto do navegador)
 - ☒ Inventário de arquivos completo, incluindo código morto identificado (`theme.js`, funções não usadas em `gemini-ai.js`)
 - ☒ Dependências mapeadas (só 2 libs via CDN + APIs externas; sem npm)
 - ☒ Secrets: nenhum secret real exposto neste documento; status CONFIGURADA/NÃO CONFIGURADA declarado
 - ☒ APIs: confirmado que não há Edge/Serverless Functions (arquitetura 100% client→serviço externo)
 - ☒ RLS: todas as tabelas mapeadas, nenhuma tabela sem RLS
 - ☒ Banco de dados: schema completo das 5 tabelas documentado
 - ☒ Performance/CLS: técnicas implementadas documentadas; métricas reais marcadas como não medidas
 - ☐ **Teste visual ao vivo em Desktop** — pendente (bloqueio de navegação `file://` nesta sessão + site publicado não visitado neste turno)
 - ☐ **Teste visual ao vivo em Mobile/dispositivo real** — pendente
 - ☐ **Hero, drawer, carrinho, footer testados clicando no site publicado** — pendente
 - ☐ **Formulários testados enviando dado real e conferindo chegada no Supabase** — pendente
 - ☒ Segurança: XSS mitigado e verificado em código
 - ☐ **LGPD: revisão jurídica formal do conteúdo das páginas legais** — não realizada, recomendada
 - ☐ **SEO: sitemap.xml, robots.txt, Open Graph, schema.org** — não implementados, recomendados como próximo passo
 - ☐ **Acessibilidade: auditoria automatizada (Lighthouse/axe)** — não realizada, recomendada
 - ☐ **Cross-browser (Firefox/Safari/Edge)** — não testado
 - ☒ Deploy: comandos reais documentados e funcionais (`push.bat/deploy.sh`)
 - ☒ Variáveis de ambiente: confirmado que o projeto não usa `.env` (arquitetura by-design)
 - ☐ **Backup formal do banco confirmado no plano Supabase ativo** — não verificável a partir do repositório
 - ☒ Procedimento de reconstrução do zero documentado passo a passo (Seção 29)
 - ☒ Histórico de correções (XSS, LGPD, componentização, dark mode revertido) registrado com evidência
-

34. Resumo Executivo Comercial

A QRV Artigos Táticos opera hoje sobre uma base técnica sólida para uma loja virtual de pequeno/médio porte: frontend rápido de publicar (site estático, sem servidor próprio para manter), banco de dados gerenciado com controle de acesso robusto por linha (RLS), painel administrativo funcional com apoio de inteligência artificial para acelerar o cadastro de produtos, e um canal de atendimento automatizado com personalidade própria.

O maior ponto de atenção arquitetural — comum a qualquer site 100% estático sem backend — é que uma das chaves de API de IA precisa, por necessidade técnica, ficar acessível ao navegador de qualquer visitante; isso foi identificado, documentado e mitigado com controles operacionais (restrição por domínio e cota diária), mas não pode ser eliminado sem introduzir uma camada de servidor no futuro.

As áreas com maior espaço de evolução, sem serem bloqueadores para operar comercialmente, são: SEO técnico avançado (compartilhamento em redes sociais sem preview rico hoje, por falta de Open Graph), ausência de qualquer barreira contra spam automatizado nos formulários, e a falta, até o momento, de medição real de performance e acessibilidade em ambiente de produção — o código aplica boas práticas, mas isso ainda não foi confirmado com números reais.

Em segurança e conformidade legal — as duas frentes de maior risco originalmente identificadas — o trabalho de correção foi completo e verificável: a vulnerabilidade de XSS armazenada foi eliminada, e a base de conformidade com a LGPD (política de privacidade, termos de uso, consentimento em formulários) foi implementada de forma tecnicamente correta, com a ressalva honesta de que isso não substitui uma revisão jurídica formal.

35. Versão Reduzida para Cliente

(documento independente, ver arquivo RESUMO-COMERCIAL-QRV.md, gerado em conjunto com este dossiê)

36. Veredito Final

Nota Desktop: 8,3 / 10 Nota Mobile: 8,1 / 10

Conclusão geral: PRONTO PARA USO COMERCIAL, COM RESSALVAS.

O site pode operar comercialmente hoje — a segurança e a conformidade legal básica, que eram os riscos mais sérios, estão corrigidas e verificadas em código. A ressalva não é sobre bloqueadores, é sobre maturidade: existem lacunas reais e concretas (SEO técnico, rate limiting, medição de performance/acessibilidade, teste ao vivo em navegador real) que não impedem a venda, mas que separam este projeto de um “e-commerce 100% maduro” no sentido mais completo do termo.

O que foi validado (comprovado pelo projeto atual e/ou pelo histórico da conversa)

- Eliminação da vulnerabilidade de XSS armazenado no painel admin.
- Implementação de RLS coerente e sem lacunas óbvias em todas as tabelas.
- Base de conformidade técnica com a LGPD (política, termos, consentimento).
- Eliminação da duplicação estrutural de header/footer via componentização própria.
- Arquitetura de IA funcional em dois pontos distintos (chat e autofill), com sanitização de saída via DOMPurify.
- Ausência de secrets de alto risco (Service Role Key) expostos no frontend.

O que foi corrigido durante o desenvolvimento (problema → solução, com evidência)

- XSS armazenado (Seção 5.1), ausência de LGPD (5.2), duplicação de header/footer (5.3), chave de API hardcoded no código (5.4/5.5), Dark Mode implementado e depois revertido a pedido do usuário (5.7), otimizações de imagem mobile (5.8), e uma extensa série de ajustes finos de UI na página de produto, incluindo a correção de uma colisão de classe CSS identificada como causa raiz de um bug visual (5.9).

O que deve ser monitorado/tratado no futuro

- Confirmar, no painel Supabase real, que supabase-admin-gemini-key.sql foi de fato executado (a policy pública ampla de site_config precisa estar substituída).
- Confirmar, no Google Cloud Console, que a chave do chat tem restrição de HTTP Referrer e cota diária ativas.
- Implementar alguma barreira anti-spam nos 4 formulários públicos.
- Adicionar Open Graph, schema.org, sitemap.xml e robots.txt — ganho relativamente barato de implementar frente ao impacto em SEO e compartilhamento social.
- Rodar uma auditoria real de performance (Lighthouse/PageSpeed) e acessibilidade (axe) no ambiente publicado, e tratar os achados.
- Definir uma rotina de retenção/expiração para os dados de mensagens_contato, solicitacoes_bordado, solicitacoes_revenda.

- Corrigir o código morto identificado (theme.js, funções não usadas em gemini-ai.js, nome de modelo gemini-3.6-flash questionável) para reduzir dívida técnica de manutenção futura.
- Realizar o teste visual ao vivo (desktop e mobile, no site publicado) que não foi possível executar nesta sessão por limitação da ferramenta de navegador disponível.

QRV Artigos Táticos — Resumo Técnico Comercial

O que o site faz e para quem

A QRV Artigos Táticos é uma loja virtual especializada em equipamentos e artigos táticos e militares: vestuário, calçados, mochilas, brevês e insígnias, cutelaria, kits e acessórios, com serviço adicional de bordado personalizado sob encomenda. O site atende consumidores finais e também recebe cadastro de revendedores parceiros. A compra é iniciada no site e finalizada em atendimento direto pelo WhatsApp.

Stack usada, em linguagem simples

O site é construído com tecnologias web padrão (HTML, CSS e JavaScript), sem depender de frameworks pesados nem de um servidor próprio para manter — isso torna a manutenção mais simples e a hospedagem mais barata e estável. Os dados dinâmicos (catálogo de produtos, mensagens de clientes, pedidos de bordado e de revenda) ficam guardados em um banco de dados na nuvem (Supabase), com regras de segurança que garantem que cada visitante só acesse exatamente o que deveria. O site conta ainda com um assistente virtual de atendimento com inteligência artificial (o “Recruta QRV”) e uma ferramenta de IA que ajuda a cadastrar produtos mais rápido no painel administrativo.

Pontos fortes reais

Segurança: a loja passou por uma auditoria dedicada que encontrou e corrigiu uma vulnerabilidade real no painel administrativo (um tipo de falha conhecida como XSS armazenado); hoje esse ponto está corrigido e verificado. O acesso ao banco de dados é protegido por regras de segurança em nível de linha, o que impede que um visitante comum leia ou altere dados que não são dele.

Privacidade (LGPD): o site tem Política de Privacidade e Termos de Uso próprios, com identificação oficial da empresa, e todos os formulários que coletam dados pessoais exigem consentimento explícito do visitante antes do envio.

Responsividade: o layout foi ajustado especificamente para boa exibição em computadores e celulares, incluindo imagens de destaque dedicadas para tela pequena e um menu lateral próprio para mobile.

Manutenção: o cabeçalho e o rodapé do site foram unificados em um único ponto de controle — qualquer alteração de menu ou rodapé no futuro é feita uma vez só e reflete automaticamente em todas as páginas.

Nota final honesta

Com base no que foi efetivamente testado e revisado até aqui: **Desktop 8,3/10** e **Mobile 8,1/10**. A base de segurança e conformidade legal está sólida. O que ainda falta para uma nota mais alta não são problemas graves, e sim itens de maturidade — como preparar melhor o site para aparecer bem quando compartilhado em redes sociais e no WhatsApp, adicionar uma proteção simples contra spam nos formulários, e confirmar com números reais (não apenas boas práticas de código) a velocidade de carregamento em uso real. Nenhum desses pontos impede o funcionamento comercial da loja hoje.

A QRV Artigos Táticos está no ar com uma base técnica confiável e corrigida nos pontos que mais importam — segurança e conformidade — com espaço claro e mapeado para evoluir ainda mais.